Hybrid Quality Control Flagging Machine Learning Model for the University of Oregon's Solar Radiation Monitoring Lab

Kalel Chester, Josh Peterson, Andrew Muehlisen

March 2026

Introduction

The University of Oregon's Solar Radiation Monitoring Lab (SRML) has long served as a center for climatological research, operating a network of meteorological data tracking stations across the Pacific Northwest. At the core of its operations, the SRML collects high-frequency data on Global Horizontal Irradiance (GHI), Direct Normal Irradiance (DNI), and Diffuse Horizontal Irradiance (DHI). Accurate, continuous solar irradiance data is critical for a diverse set of real-world applications, ranging from predictive solar energy forecasting and power grid load management to long-term climatological research and localized atmospheric studies [1]. However, ensuring the highest standard of data quality within these continuous data streams is inherently challenging.

Traditional quality control (QC) methodologies have predominantly relied on manual, human-driven inspection combined with simple, rule-based threshold algorithms [7]. While functional, these conventional techniques are highly time-consuming, prone to human fatigue, and often struggle to capture the complex, multi-variate patterns characteristic of partial sensor failures. As the volume of data collected by the SRML grows and it transitions to a more automated operational model, the need for a scalable, automated validation framework becomes increasingly apparent.

This capstone project seeks to address these limitations by developing an automated machine-learning workflow tailored for QC flagging of SRML's solar irradiance data. The objective is to explore how modern, data-driven techniques can augment physical sensor networks and significantly reduce the manual burden on analysts. By implementing a pipeline hybrid model architecture that combines a Random Forest, an Isolation Forest, and an Attention-based Recurrent Neural Network (RNN), we enable the system to both detect known hardware failure modes and identify previously unseen anomalies by contextualizing data in a time-series feature space.

Methods

Our methodology utilizes a pipeline that ingests raw station data, generates astronomically derived clear-sky expectations, and augments the training set with realistic, synthetic sensor errors. These engineered features are fed into our isolation- and random-forest ensemble, before being fed into our RNN to produce final quality probability scores. Finally, a manual quality-control application enables a human-in-the-loop review and model-training system that maintains expert oversight while iteratively improving the model's predictive power.

Data Collection and Preprocessing

The primary dataset for this study originates from the SRML's Seattle tracking station (STW) and spans multiple years of high-frequency records (January 2023–February 2026). Measurements of GHI, DNI, and

DHI are collected alongside various meteorological parameters, including temperature, solar zenith angle, and solar azimuthal angle at 1-minute intervals. Physically, GHI is measured with a fixed horizontal pyranometer, while DNI and DHI are measured with a tracking pyrheliometer and a shadowband setup, respectively.

Training data for the model consisted of three years of 1-minute-interval records from 2023 through 2025, excluding months dominated by broken-tracker errors (April 2024 and January 2025). This data was supplemented with a 2:1 real-to-error-injected ratio to increase the number of "bad" data points. This process is further explained below.

Test data used for final model evaluation includes a clean, manually flagged version of the first two months of 2026, along with a copy of February 2025 through February 2026 with artificial errors inserted. August 2025 was excluded due to a persistent failure that rendered the data unusable for testing.

Because the project commenced with a vast quantity of unflagged historical data, an iterative human-in-the-loop approach was adopted. Initially, a subset of the data was manually labeled by analysts to form a foundational training set. Models trained on this initial set were then deployed to predict labels for the remaining unflagged data, which analysts subsequently reviewed and corrected. This human-in-the-loop, cyclical training process gradually expanded the ground-truth dataset and enhanced the model's robustness.

To extract meaningful signals for the machine learning models, a custom data pipeline was engineered. The pipeline utilizes the physics library "pvlib" [2] to calculate clear-sky irradiance models for the Seattle site. These physically derived expectations serve as baseline comparisons for the actual sensor readings, allowing the models to evaluate irradiance deviations dynamically without arbitrarily altering the native timestamps.

The features engineered in preprocessing to support the model with more meaningful and physically interpretable information are summarized below:

Table 1: Engineered Features Summary

Category	Feature Name	Description
Temporal	Timestamp_Num	Unix epoch seconds for numeric model compatibility.
	hour_frac	Fractional hour of the day (e.g., 14.5 for 2:30 PM).
	hour_sin, hour_cos	Cyclical encoding of the time of day.
	doy_sin, doy_cos	Cyclical encoding of the day of the year (seasonality).
Solar Geometry & Clear-Sky	elevation, SZA	Solar elevation and Solar Zenith Angle (degrees).
	GHI_Clear, DNI_Clear, DHI_Clear	Clear-sky irradiance estimates modeled via pvlib.
	CSI	Clear-sky index (ratio of measured GHI to GHI_Clear, clipped to [0,3]).
Consistency & Ratios	ghi_ratio, ghi_diff	Ratio and absolute difference between measured GHI and calculated GHI.
	dni_clear_ratio, dhi_clear_ratio	Ratio of measured DNI and DHI to their respective clear-sky estimates.
	dni_ghi_ratio, dhi_ghi_ratio	Ratio of direct and diffuse components to global horizontal irradiance.
	beam_horizontal_consistency	Consistency check between horizontal beam component and GHI - DHI.
Physical	QC_PhysicalFail	Binary indicator (0 or 1) flagging physical impossibility or severe closure failures (BSRN-inspired).

Data Augmentation and Synthetic Error Injection

A significant challenge in predictive quality control is the severe class imbalance inherent to sensor data; genuine hardware failures are relatively rare compared to periods of normal operation. To ensure that the models learn robust decision boundaries for anomalous events, a synthetic error-injection module was developed. Realistic sensor faults—such as simulated sensor-cleaning events, reduced feature redouts caused by dust accumulation or shade, and moisture-magnification effects like water droplets—were procedurally introduced into the training data. The number of errors per file was selected randomly from a uniform distribution over 30 to 90 injected errors per month, with durations randomly sampled from a distribution custom-tailored to simulate the range of physical durations for each custom error function. This resulted in an error frequency ranging from about 1 to 3 injected errors per day. These injections were context-aware, using solar zenith angles to ensure that errors such as droplet magnification appeared only at physically appropriate times.

A summary of the error functions and associated selection weights is listed below:

- **Reduce features — weight 39%:** This function randomly selects one or more solar features (from GHI, DNI, DHI) and reduces their values by a percentage sampled uniformly from 6% to 50%. The duration is sampled from a uniform distribution over 1–30 minutes, and the reduction profile is applied using a random draw between a box (square) window and a Gaussian window.
- **Water droplet — weight 20%:** This function randomly selects a single target feature from GHI or DNI, then increases that feature by a uniformly sampled spike of 5% to 25% to simulate lensing/magnification from droplets. The duration is shorter, sampled from a uniform distribution over 1–5 minutes.
- **Copy from another day — weight 20%:** This function randomly selects one or more solar features (from GHI, DNI, DHI) and replaces values with data taken from the same clock time on a randomly offset prior day (offset sampled uniformly from -30 to -1 days). Event duration is drawn from a uniform distribution over the 3–30-minute range.
- **Cleaning event — weight 20%:** This function simulates servicing by sequentially reducing the target features over a 3-minute event (one feature per minute), with each reduction sampled uniformly from 4% to 8%.
- **Morning frost — weight 1%:** This function randomly selects one or more features (from GHI, DNI, DHI) and applies a sunrise-intensifying frost response over the event window: DNI decreases, DHI increases slightly, and GHI increases strongly. The base frost magnitude is sampled uniformly from 10% to 40%, and the effect ramps up through the event duration (currently sampled as 30–90 minutes in implementation).

This error injection updated the class weights for the 2025 data as seen below:

Table 2: Class Weights for 2025 Data Post-Injection

Target	Regular Good [%]	Regular Bad [%]	Injected Good [%]	Injected Bad [%]
GHI	99.36	0.64	97.68	2.32
DNI	99.67	0.33	97.84	2.16
DHI	99.59	0.41	98.01	1.99
Combined	99.53	0.47	97.84	2.16

Hybrid Machine Learning Architecture

The core of the QC system is a pipeline architecture built in Python using "scikit-learn" [9] and JAX/Flax [3, 6, 4]. This architecture processes the engineered features through three complementary models, wherein the random forest and isolation tree feed into the RNN for the final model output:

- **Random Forest Classifier:** Serving as the baseline supervised learner, a Random Forest with 64 trees captures complex, non-linear interactions between the different irradiance variables and clear-sky deviations. It uses class weighting to handle class imbalance and applies probability calibration to produce reliable confidence scores.
- **Isolation Forest:** Functioning as an unsupervised anomaly detector, an Isolation Forest with 128 trees is trained exclusively on validated "good" data. Its primary role is to act as a safety net, calculating anomaly scores to flag novel, out-of-distribution failure modes that were neither present in the human-labeled data nor generated by the synthetic error injections.
- **Attention-Based Recurrent Neural Network (RNN):** To capture the time-series nature of daily solar cycles, a deep learning component was constructed using "JAX" and "Flax" [3, 6, 4]. The network employs a two-layer Gated Recurrent Unit (GRU) with a hidden dimension of 64 and two stacked layers with dimensions of 64 and 32, analyzing continuous 60-step sequence windows (representing one hour of 1-minute data). An integrated temporal attention mechanism enables the network to automatically learn to focus on the most critical time steps, such as rapid cloud-cover shifts or dawn/dusk transitions. Dropout regularization (rate 0.2) and residual connections are used to prevent overfitting and ensure stable gradient flow during training.

Deployment and Human-in-the-Loop Integration

The entire workflow is orchestrated via automated learning scripts. Once trained, the system performs inference on unseen station data. Importantly, the prediction module defaults to a safe-write philosophy: it writes machine learning predictions back to the output CSVs while strictly preserving any existing manual "bad" quality flags, ensuring that prior human validation is never overwritten. Finally, a custom-built graphical user interface (GUI) empowers SRML analysts to visually inspect the model's classifications, review and navigate to least certain underlying probability scores as defined by the model, and easily adjust target flags. This deployment shifts the human's role from low-level data scanning to high-level model supervision.

Seasonal Performance

Because the model is designed to capture temporal patterns, it is crucial to evaluate its performance across the different time resolutions at which it operates. To do this, we can view time-series diagnostics that plot the model's predicted anomaly rates across hours of the day and months of the year, comparing them to the actual error frequencies (Figures 3 and 4). This allows us to assess whether the model is well-calibrated across different temporal contexts and whether it captures known patterns of sensor behavior, such as increased anomalies during dawn/dusk transitions or seasonal variations in sensor performance.

Results

Our hybrid machine learning model for quality-control flagging on the SRML's solar irradiance data demonstrates significant improvement over traditional rule-based methods, though it has not yet reached a performance level sufficient to warrant consideration for autonomous, unsupervised operation.

Performance varied significantly depending on the target irradiance metric. The model achieved the highest performance on the Global Horizontal Irradiance (GHI) data, exhibiting strong classification abilities (F1 scores of 0.83 for regular data, and 0.88 for error-injected data). Conversely, identifying anomalies in Direct Normal Irradiance (DNI) proved to be the most challenging, resulting in lower performance (F1 scores of 0.63 for regular data and 0.70 for error-injected data). The DHI model showed performance closer to the GHI model for the regular data (F1 score of 0.80), and a F1 score closer to the DNI model for the error-injected data (0.75). The results for each model are summarized in Table 3, which includes the models' accuracy, precision, recall, and F1 scores.

Table 3: Model Performance Metrics for GHI, DNI, and DHI

Dataset	Feature	Samples	Agreement	Accuracy	Precision	Recall	F1
Regular	Flag_GHI	84,835	84810/84835	0.9997	0.9833	0.7108	0.8252
Regular	Flag_DNI	84,848	84777/84848	0.9992	0.9242	0.4803	0.6321
Regular	Flag_DHI	84,725	84709/84725	0.9998	1.0000	0.6667	0.8000
Injected	Flag_GHI	458,658	457840/458658	0.9982	0.9339	0.8281	0.8778
Injected	Flag_DNI	427,156	425919/427156	0.9971	0.8245	0.6079	0.6998
Injected	Flag_DHI	428,560	427401/428560	0.9973	0.8503	0.6669	0.7475

Confusion Matrices

To further understand the classification behavior across the three irradiances, we generated target-level confusion matrices (Figure 1). For regular files, DNI exhibits a high false-negative rate, overpredicting "good" data when humans marked an anomaly. DHI shows a higher false positive rate, flagging human-labeled good data. GHI shows a more balanced distribution, with a stronger true classification rate and minimal off-diagonal spread compared to the other models. The injected error matrices demonstrate that when synthetic anomalies are introduced, the model struggles to accurately capture them across all targets compared to the regular data. This suggests that the model is better trained to identify realistic, often more obvious anomalies that humans are more likely to identify.

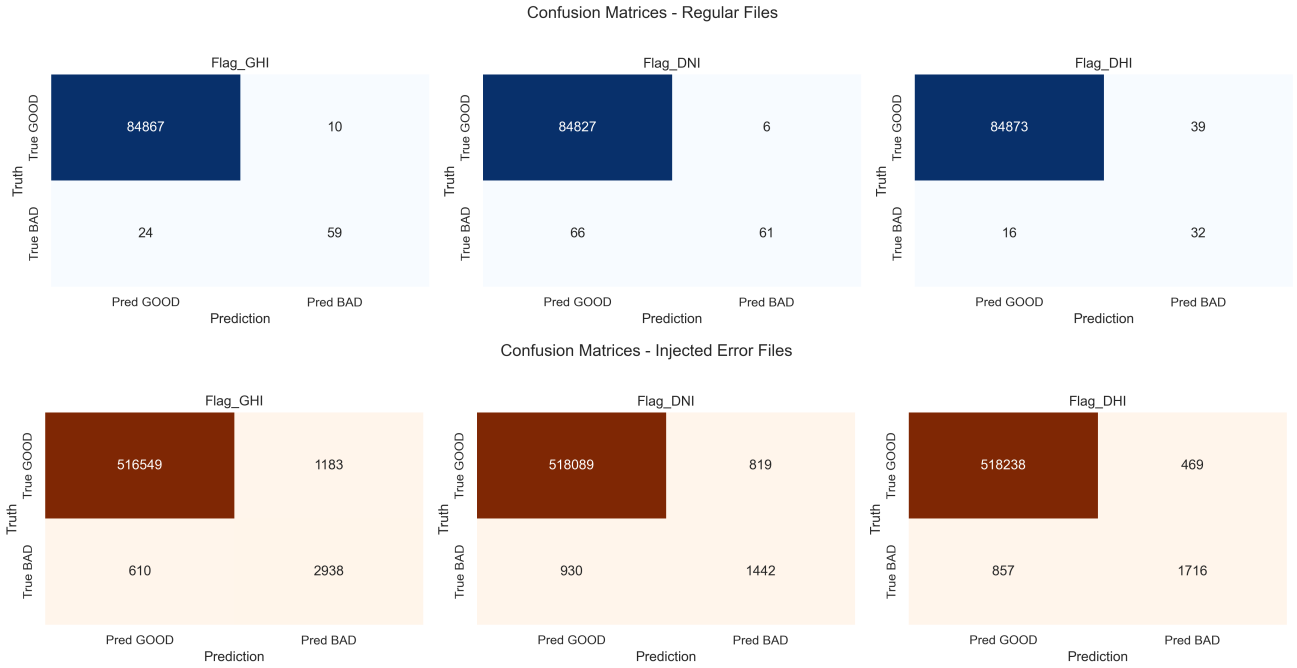

Figure 1: Target-Level Confusion Matrices for GHI, DNI, and DHI

ROC Curves

The ROC curves for each target are depicted in Figure 2, illustrating the trade-off between true positive rates and false positive rates. The curves clearly illustrate that the Global Horizontal Irradiance (GHI) model exhibits the strongest bow toward the top left, achieving the highest Area Under the Curve (AUC) across both datasets (0.991 for regular data, and 0.985 for error-injected data). Diffuse Horizontal Irradiance (DHI) follows closely behind (0.990 for the regular data, and 0.955 for the error-injected data), while Direct Normal Irradiance (DNI), yields a noticeably shallower curve (AUC of 0.956 for the regular data, and 0.922 for the error-injected data). This reflects the DNI model's struggle to separate transient noise from true tracking anomalies. Furthermore, comparing the regular files with the injected-error files reveals wider, cleaner AUC margins for the injected data, confirming the pipeline's greater sensitivity to real, non-synthetic faults.

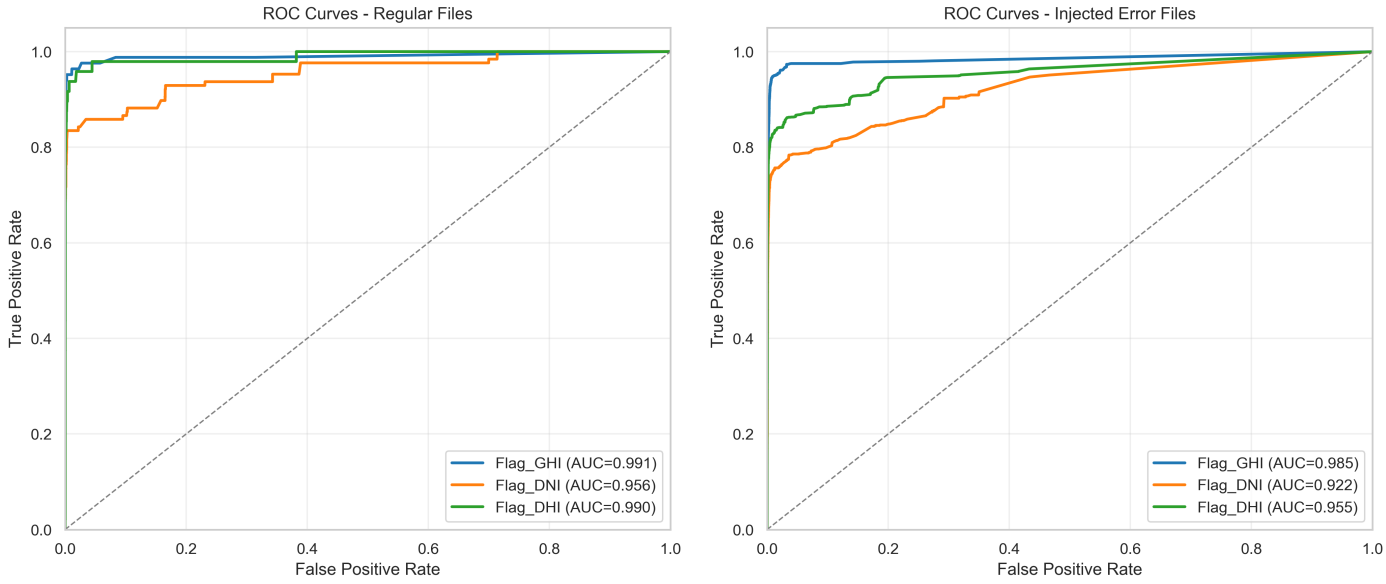

Figure 2: Target-Level ROC Curves

Time-Series Diagnostics

Our time-series diagnostic plots (Figure 3 and Figure 4) reveal that the model's daily predicted anomaly rates align well with actual error frequencies, with one exception in GHI around 19:00. The monthly diagnostics show that the models perform relatively consistently for all seasons, matching the slightly noisy fit seen in the hourly diagnostics.

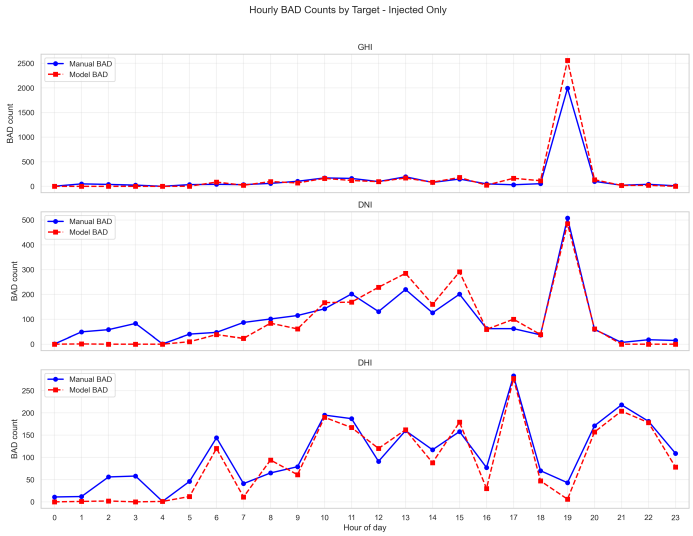

Figure 3: Hourly Error Distribution

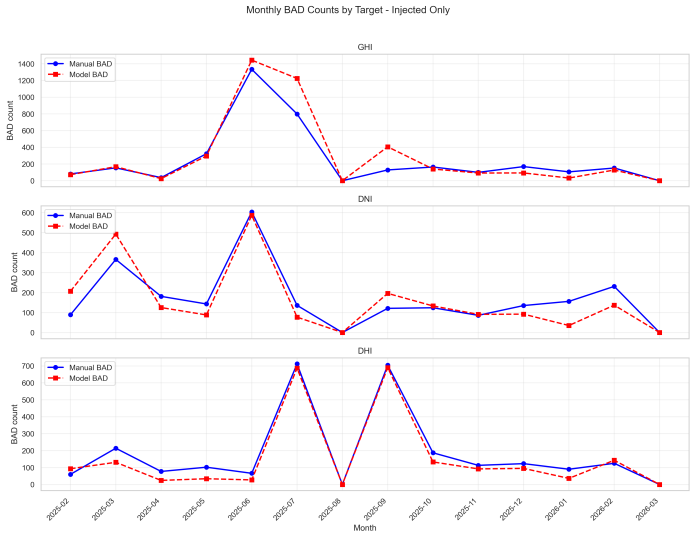

Figure 4: Monthly Error Distribution

Discussion

The results highlight both the potential and the inherent complexities of applying automated machine learning to meteorological quality control. Our goal was to augment the existing human-driven workflow by modeling multivariate and temporal anomalies. The performance of the GHI and DHI flagging validates our hybrid architecture's ability to utilize clear-sky baseline expectations and synthetic error injections to establish robust decision boundaries for stable physical metrics.

However, we can see that there is still disparate performance between our models, with the GHI model consistently outperforming the DHI model, which in turn out performs the DNI model. These relative performance metrics can largely be attributed to the quality of the data and the anomaly-detection supporting features for each target. The GHI model consistently performs best, except for its tendency to overpredict anomalous points around 19:00, as seen in Figure 3. This is likely because the model is overfitting to a specific pattern in the training data, with more anomalies during this summer evening period. This could be caused by a variety of factors, such as an increased number of obstructions (e.g., telephone poles and trees) or environmental conditions that affect the GHI sensor's performance during this time.

The comparatively poor performance of the DNI and DHI models can be attributed to the greater difficulty of identifying errors in these features. Because DNI requires a tracking pyrheliometer to face the sun directly, it is highly sensitive to subtle mechanical tracking misalignments, transient cloud edges, and atmospheric aerosols. These create volatile data signatures that often mask genuine anomalies. The physical requirement for precise alignment causes the DNI sensor to produce sharp, high-frequency signal drops due to micro-misalignments, which the model struggles to distinguish from actual sensor failures, leading to elevated error rates (Figure 1).

Meanwhile, anomalous DHI values are more challenging to identify because of their comparatively low magnitudes and the difficulty of translating them into obvious anomaly-signaling features. This leads the DHI model to tend towards identifying more false positives in the regular data. Because of this, the model is designed to take a more conservative approach, erring on the side of caution when flagging points as anomalies. We can see that this behavior differs between the regular and error-injected datasets, with the model predicting more false positives on the regular data and more false negatives on the error-injected data.

While the hybrid architecture demonstrates strong empirical performance, we did not perform a full

ablation study isolating each component. Future work could quantify the marginal contribution of the Isolation Forest and temporal RNN components relative to the Random Forest baseline.

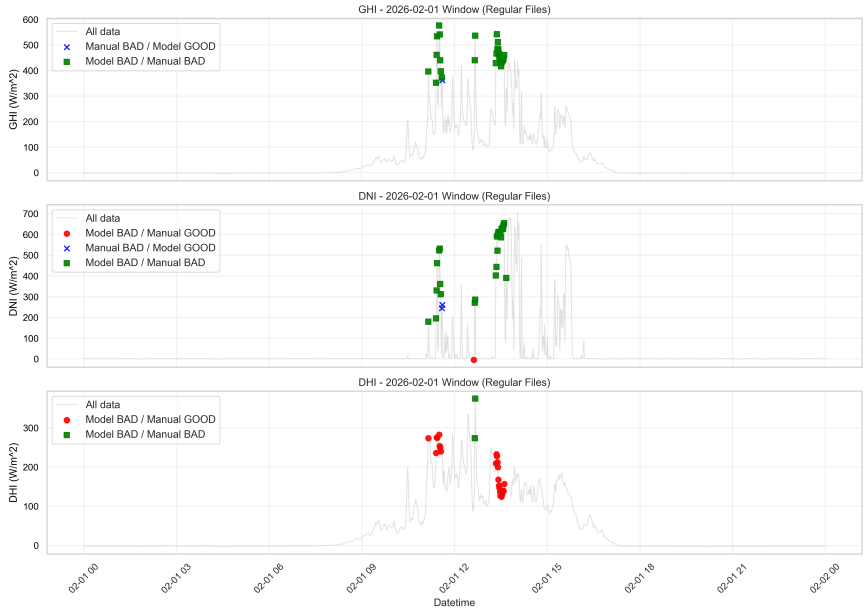

Figure 5: Misaligned Manual vs. Model-Identified Anomalies

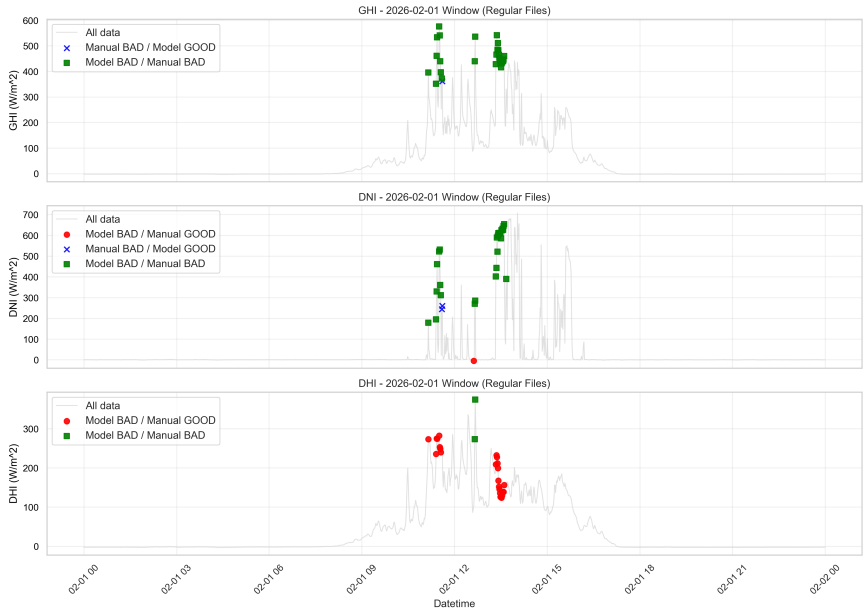

Figure 6: Misaligned Manual vs. Model-Identified Anomalies

It is worth noting that although the model's performance, as measured by traditional metrics, is insufficient for unsupervised deployment, its practical utility in a human-in-the-loop context remains significant. While the model does not always capture the full temporal range or correct causal features of an anomalous event, almost all events, marked by one or more "bad" points, are correctly identified in some way. In Figures 5 and 6, we can see two examples of such days, comparing model predictions to human labels.

In the first example (Figure 5), the model correctly identifies a cluster of anomalous points in the GHI and DNI data, but it incorrectly flags the DHI data as anomalous as well. This highlights an inherent problem with these methods: while many anomalous events are easily recognized, identifying the specific

irradiance feature at fault can be challenging and subjective. Further review of the data suggests that there are many points at which the model's predictions are likely to be less subjective than those of analysts. This is because, in this data, human analysts are seen to have a preference towards marking a "responsible feature" as bad (e.g., having a preference towards marking DNI bad when an anomalous event has ambiguity in its causal feature), even when it is not clear that it is responsible for the anomaly. This process is highly prone to human error, as the same event has been marked differently by the same analyst on different days. Meanwhile, at these same points, the model is more likely to flag all potentially responsible features as bad, leading to more consistent results.

In the second example (Figure 6), the model again identifies a cluster of anomalies, but it fails to identify the proper bounds and responsible features. This suggests that while the model is effective at identifying potential anomalies, it may also be prone to false negatives, particularly when the underlying data is noisy or the anomalies do not conform to typical patterns. By providing a GUI that allows analysts to easily review predicted probabilities and adjust flags, the model can still substantially reduce the manual burden on researchers. Even if the model fails to capture all anomalies, it can still be valuable in guiding analysts to the most uncertain data points, allowing them to focus their attention where it is most needed. This demonstrates the importance of considering the practical application of machine learning models in real-world contexts rather than relying solely on traditional performance metrics to evaluate their utility. While this model does not identify "bad" points accurately enough to support its unsupervised function, the guidance it offers analysts makes it an invaluable tool.

The greatest limitations to this project's objectives lie in the scope and complexity of synthetic error injections and in the quality of the manually flagged data used for model training. While simulating dust accumulation and droplet magnification was effective for stationary sensors, it may not have adequately represented the complex errors unique to tracking sensors. A more sophisticated physical simulation of tracking failure modes could improve the DNI recall. Additionally, the model's performance is greatly limited by the quality of the training data. Because of the noisy nature of the raw data and the rarity of true anomalies, the initial human-labeled dataset was relatively small and contained labeling inconsistencies. This could have led the model to learn suboptimal decision boundaries, particularly for the more volatile DNI data. A larger, more consistently labeled dataset would likely improve model performance across all targets. Lastly, while the Attention-based RNN provided temporal context, it also introduced significant computational overhead, leading to challenges in training and operating the model and potentially bottlenecking real-time inference in broader multi-station deployments.

Conclusion

This project successfully developed and deployed a hybrid machine-learning quality-control pipeline for the University of Oregon's Solar Radiation Monitoring Lab. By integrating a Random Forest, an Isolation Forest, and an Attention-based RNN, the project moved beyond traditional rule-based flagging to identify complex, time-series irradiance anomalies. While automated flagging of highly sensitive metrics such as DNI remains challenging, the system achieved high precision and excellent accuracy. Ultimately, by optimizing for recall and wrapping the models in a custom analyst GUI, the system achieved its primary goal: substantially reducing the manual burden on researchers while ensuring expert oversight remains central to the SRML's data validation process.

References

[1] Alizamir, M., Shiri, J., Fard, A. F., Kim, S., Gorgij, A. D., Heddam, S., & Singh, V. P. (2023). Improving the accuracy of daily solar radiation prediction by climatic data using an efficient hybrid deep learning model: Long short-term memory (LSTM) network coupled with wavelet transform. *Engineering Applications of Artificial Intelligence, 123*, 106199. https://doi.org/10.1016/j.engappai.2023.106199

[2] Anderson, K., Hansen, C., Holmgren, W., Jensen, A., Mikofski, M., & Driesse, A. (2023). PVlib Python: 2023 project update. *Journal of Open Source Software, 8*(92), 5994. https://doi.org/10.21105/joss.05994

[3] Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). JAX: composable transformations of Python+NumPy programs (Version 0.3.13) [Computer software]. http://github.com/google/jax

[4] DeepMind, Babuschkin, I., Baumli, K., Bell, A., Bhupatiraju, S., Bruce, J., Buchlovsky, P., Budden, D., Cai, T., Clark, A., Danihelka, I., ... & Viola, F. (2020). The DeepMind JAX Ecosystem [Computer software]. http://github.com/google-deepmind

[5] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., ... & Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

[6] Heek, J., Levskaya, A., Oliver, A., Ritter, M., Rondepierre, L., Steiner, A., & van Zee, M. (2020). Flax: A neural network library and ecosystem for JAX [Computer software]. http://github.com/google/flax

[7] Maxwell, G., Stoffel, T., Rymes, M., & Myers, D. (1993). *Users Manual for SERI QC Software: Assessing the Quality of Solar Radiation Data* (No. NREL/TP-463-5608). National Renewable Energy Lab. (NREL), Golden, CO (United States).

[8] McKinney, W. (2010). Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 51–56).

[9] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., ... & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

[10] Vignola, F., & Andreas, A. (2013). *University of Oregon: Solar Radiation Monitoring Lab (Data).* NREL Report No. DA-5500-64452. https://doi.org/10.7799/1183467